# Plagiarism Detection and Visualization

Nate Phillips

Josh Crowson

For this project, we attempted to design a system that could take in two documents, identify any plagiarized text, and then visualize both documents and the plagiarized text. The purpose for this program is to enable end users to quickly and effectively determine if plagiarism has taken place. The goal for this visualization, then, is two-fold: to detect plagiarism and to display it in such a way that it is understandable and relevant to an end user.

To accomplish these goals, we split up our project into two distinct tasks. The first task was to examine the two documents and produce a list of plagiarized phrases. The second task was to take these phrases and highlight them, so that the end user can examine them to determine whether any plagiarism has taken place. These two pieces were written completely separately, with the first written in Python by Josh Crowson and the second written in Javascript by Nate Phillips. The javascript portion of the program takes in both the two original text documents, as well as the list of plagiarized phrases generated by the Python code.

Our plagiarism detection software takes in two input text files and individually breaks the texts into word groups of size 3-7. When it breaks up the text, it strips out the punctuation. The program then compares the word groupings from one text to the word groupings of the other text and stores union of the two sets of word groupings in another list. Next, the program reduces the list of word groupings in common between the two texts by removing redundant word groupings. An example of this reduction is to remove the word group "[b, which, eventually]" if the word group "[ritchie, modified, b, which, eventually, became]" is also a part of the list. For this project, we compared an 8 page paper and a 5 page paper. We found 80 word groupings between the 2 papers, which ended up reducing down to 12 unique word groupings.

On the visualization side of this project, we initially considered several different approaches, such as arc graphs. However, the approach we ended up sticking with was to display both text documents on a web page. Inside these documents, words were highlighted based on whether they were plagiarized and the significance of the plagiarism. Our visualization displays three different levels of plagiarism. The lowest level involves three- or four-word plagiarisms and is not necessarily significant in determining whether something is plagiarized. The next level highlights five- or six-word plagiarisms and is a more telling indication of possible plagiarism. The final level is only used for plagiarized phrases with seven or more words in them, which represent very suspicious activity that may on its own be a sufficient indicator of plagiarism. In the visualization, each of these levels is mapped to a different color. The colors are, in order of severity, dark blue, light blue, and red. This gives users something of an intuitive grasp of the likelihood of plagiarism; if there are just a few dark blue phrases, then plagiarism is unlikely. However, if there are several red or light blue phrases, then there is significant likelihood of plagiarism.

Of course, even texts with several red phrases sprinkled throughout might not be plagiarizing. The red phrases might be part of a direct quote that is appropriately cited. They might also be, at least theoretically, a simple coincidence-two authors talking at length about related subjects who just happen to phrase things in a similar way. However, these questions are ultimately up to the standards of the user; this visualization can not, on its own, determine whether a work is plagiarized. It can, however, be used to inform an end user about the relative probabilities that one work is plagiarized by another.

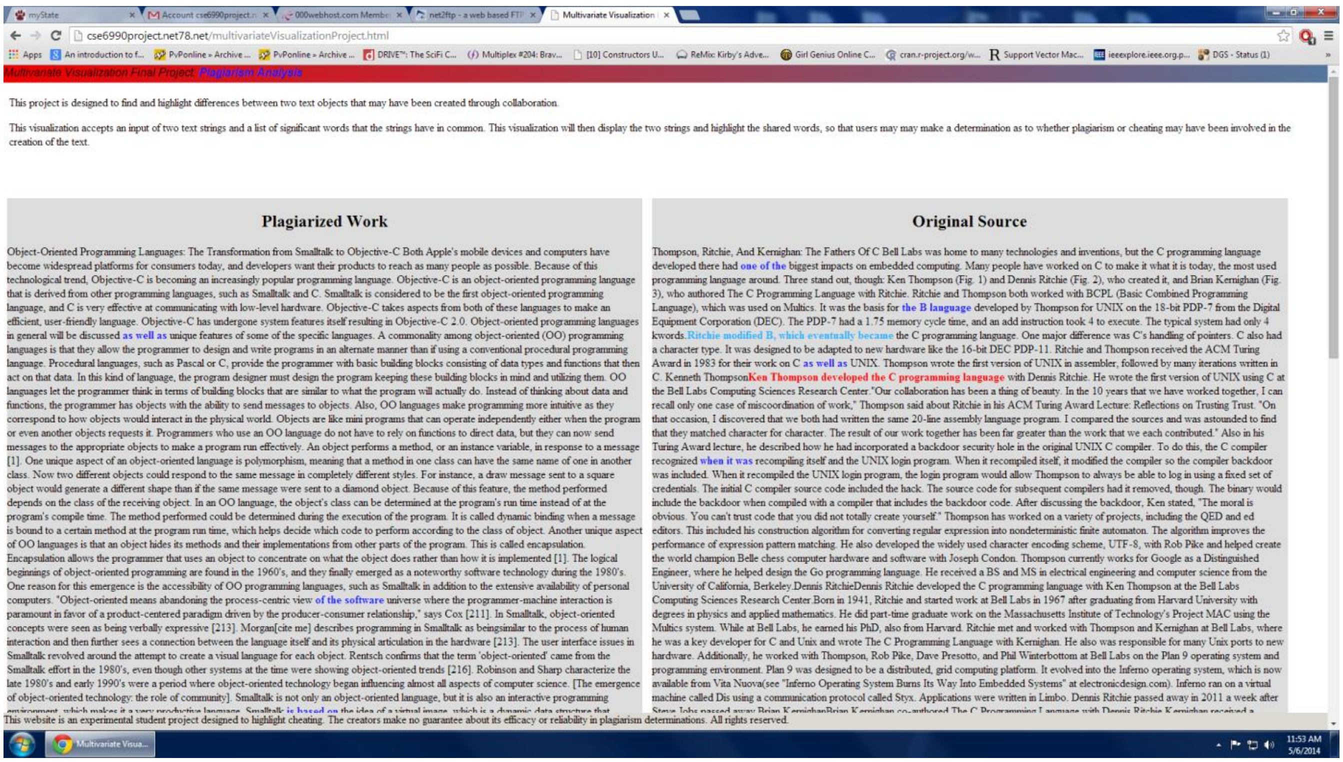


Figure 1 – An example of our visualization in action (view at the top of the page).

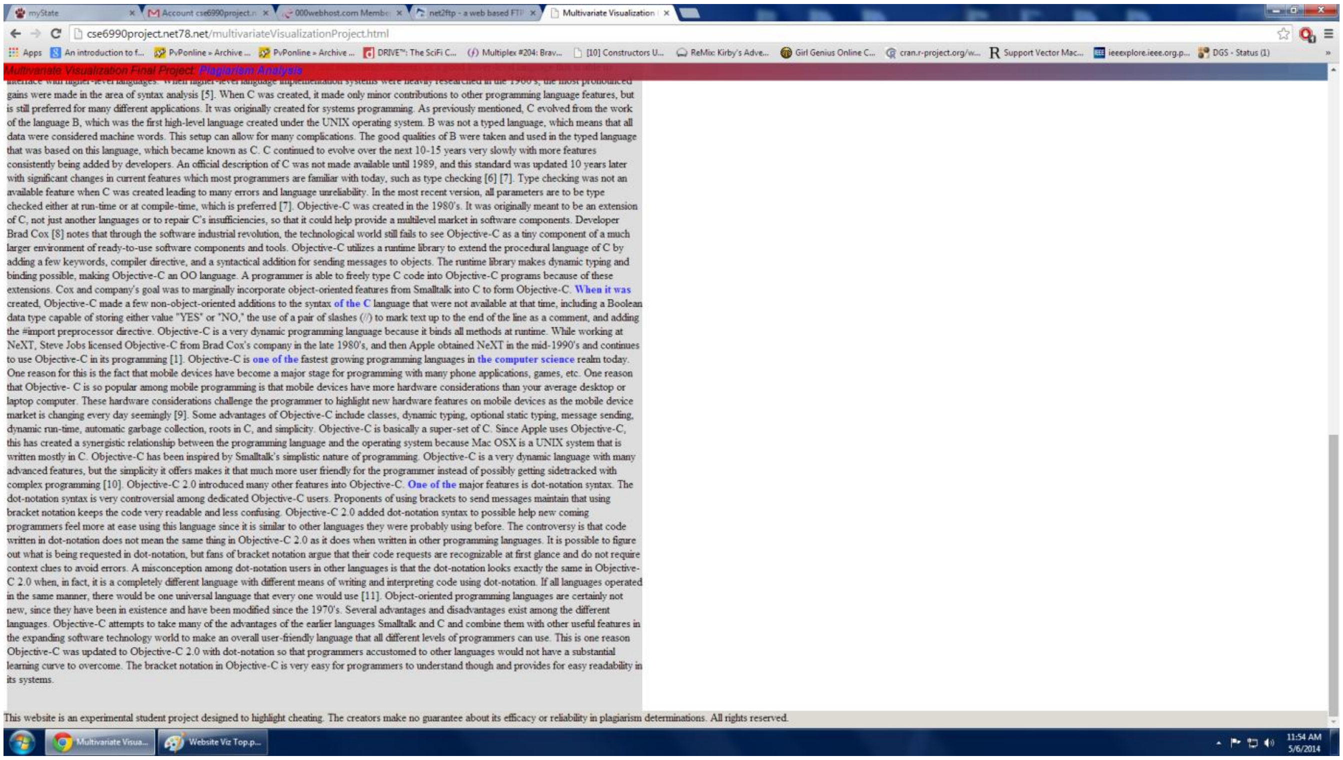


Figure 2 – An example of our visualization in action (view at the bottom of the page).